



WIA1002 DATA STRUCTURE

Smart Library Project TECHNICAL REPORT

SEM II 2025/2026

**DR. MOHAMED N. M. LUBANI
OCCURRENCE 10, TEAM 2:**

CHEN BINGYAN	24078302
BEBY AZZURA	24201647
YUDONG CHEN	24074901
HILAL MUMTAZ SALEH HARAHAHAP	24200675
NAZEEF NUWAISIR KHAN	24085530

Github Link: this is the actual URL:

https://github.com/24078302Liz/DataStructure_GroupProject_Sem2.git

Table of Contents

1. Introduction & Executive Summary	3
1.1 Team Distribution & Responsibility Matrix	4
2. Abstract Data Type (ADT) Design & Information Hiding	5
3. Catalogue Architecture: Binary Search Tree (BST)	6
3.1 Node Object Design & Field Layout	6
3.2 Recursive Insertion Optimization	6
3.3 Logarithmic Search Path Operations	6
3.4 Multi-Scenario Node Deletion (The Borrowing Trigger)	7
3.5 Full-Traversal Keyword Search (Title & Author)	7
4. Transaction Registry: Stack Design	8
4.1 Encapsulation of the Legacy Collection Layer	8
4.2 LoanRecord: Enriched Transaction Model	8
4.3 Null-Safety Boundaries & Input Defenses	9
4.4 Non-Destructive Reverse-Chronological Rendering Logic	9
4.5 Student-Specific Record Retrieval	9
5. Fine Management Engine	10
5.1 Fine Calculation Logic	10
5.2 Balance Tracking with HashMap	10
5.3 Payment & Display Operations	11
6. System Integration & Console UI Layer	11
6.1 Atomic Borrowing Lifecycle Flow	11
6.2 Extended Search Submenu	12
6.3 Fine Manager Submenu	12
6.4 Robust Type-Parsing & Error Recovery Scanners	13
6.5 Overdue Record Demonstration Utility	13
7. Algorithmic Complexity & Performance Analysis	14
7.1 Data Structure Operation Complexity Matrix	14
8. Verification Matrix & Quality Assurance Summary	15
8.1 Comprehensive Quality Assurance Testing Ledger	15
9. Conclusion	17

1. Introduction & Executive Summary

The Smart Library Management System is a terminal-based tool built to handle the everyday administrative work of a university library. Rather than reaching for off-the-shelf solutions, the project was designed from the ground up using object-oriented principles, partly as a practical exercise, and partly to answer a genuine question: what happens when you pit different data access strategies against each other inside the same system?

One of the core tensions in any library software is that not all data behaves the same way. The catalogue needs to answer the question “Does this book exist?” almost instantly, no matter how large the collection grows. Transaction records, on the other hand, need to preserve a reliable, chronological trail of who borrowed what and when. These two requirements pull in opposite directions, and bridging them is where things get interesting.

To handle this, the system uses a dual-engine approach where each layer of data is managed by a structure suited to its specific job. The system has since been extended with three additional capabilities: multi-field catalogue search (by title and by author), a structured LoanRecord model that enriches each borrowing transaction with student identity and timestamp data, and a dedicated FineManager subsystem that computes, tracks, and clears outstanding student fine balances.

- **The Catalogue Inventory:** The book catalogue is managed entirely through a custom Binary Search Tree. This means that whether you are adding a new title or searching for one by ISBN, the system does not have to scan through everything it knows. It narrows down to the right answer in logarithmic time, so even as the collection grows deeper, lookups stay fast. The catalogue additionally supports full-traversal keyword searches by title and by author, providing flexible discovery beyond strict ISBN matching.
- **The Borrowing Registry Ledger:** Governed by a stack, the most recent activity always surfaces first in the registry. Each entry is represented by a LoanRecord object that captures not just the book reference, but also the student identifier and the exact date of the transaction.
- **The Fine Management Engine:** A dedicated FineManager class calculates overdue fines based on a configurable daily rate and standard loan period, maintains per-student outstanding balances using a HashMap, and exposes payment and balance-inquiry operations through the console UI.

To keep the modules working together without them becoming too tangled up in each other’s internals, the team introduced an Abstract Data Type (ADT) interface layer as a kind of shared contract between components. The rest of this report walks through how the system was built, how each class fits together, how the algorithms perform, and how the whole thing held up under testing.

1.1 Team Distribution

Building the Smart Library was a team effort, with each member taking ownership of a specific part of the system so work could move forward in parallel without stepping on each other's toes:

Engineer	Role Assignment	Core Architectural Responsibility
Nazeef	Catalogue Architect	Engineered the node topology (Book class) and structural BST injection layer; developed the recursive leaf and branch node balancing insertion mechanics.
BingYan	Borrowing History	Constructed the BorrowStack transactional layer; enforced strict null-safety boundaries and implemented non-destructive reverse-index stack traversal. Extended the stack to store enriched LoanRecord objects and introduced the getAllByStudent() and getLatestByStudent() lookups used by the FineManager.
YuDong	Record Finder	Designed and optimized the recursive lookup algorithms within the hierarchical catalogue tree, establishing key-matching conditional branches. Extended the BST with full-traversal keyword search operations by title and by author.
Hilal	ADT Designer	Authored the structural contract (LibraryADT); governed architectural compliance, encapsulation audits, and verified global information hiding properties.
Beby	Admin Logic & Integration	Owned the admin logic for the borrowing process (removing the borrowed book from the BST catalogue and pushing its LoanRecord onto the stack). Also wrote the main loop console harness, implemented safe token-based input scanners, built the type-parsing error-recovery blocks, fused the BST-to-Stack borrowing transaction workflow, and integrated the Fine Manager submenu into the console UI layer.

2. Abstract Data Type (ADT) Design & Information Hiding

Software robustness depends on the restriction of access vectors. Following the software engineering principle of Information Hiding, the internal states of data structures must remain unexposed to client-side routines. The application guarantees this security boundary via `LibraryADT.java`, an interface defining the operational capabilities of the library backend without leaking its architectural details.

The LibraryADT Interface Specification:

```
public interface LibraryADT {  
    void addBook(int isbn, String title, String author);  
    Book searchBook(int isbn);  
    void borrowBook(int isbn, String studentId);  
    void viewLatestHistory();  
}
```

By mandating that the orchestration class (`SmartLibrary.java`) implements this interface, the client UI layer is bound to a stable contract for the core catalogue and borrowing operations. The interface captures these four foundational endpoints, while the extended capabilities added in this version (keyword search by title and author, and the Fine Manager operations) are coordinated through the same orchestration layer and delegate to their dedicated subsystems rather than exposing any internal data structure. Information hiding is therefore enforced at the data-structure level: regardless of how many operations the menu drives, no client routine can reach the BST root pointer, the stack collection, or the balance map directly. The following design constraints were systematically enforced across the codebase:

- **Encapsulated Tree Pointers:** The root node pointer in `BookBST.java` is strictly marked as `private Book root;`. Outside classes cannot traverse or reassign the root node, preventing corruption of the tree topology.
- **Protected Stack Arrays:** The underlying storage collection inside `BorrowStack.java` is defined as `private Stack<LoanRecord> stack;`. This thwarts accidental index wiping, pop overruns, or manual insertion of out-of-order logs.
- **Complete State Isolation:** The main instance references inside `SmartLibrary` – namely catalogue, history, and fineManager – are declared private. The UI layer can never directly invoke tree, stack, or fine management methods.
- **LoanRecord Encapsulation:** The `LoanRecord` class exposes its borrowing context (book, student ID, and borrow date) only through getters, preventing external mutation of transaction data once a borrowing event has been logged.

3. Catalogue Architecture: Binary Search Tree (BST)

The core collection inventory is managed by a Binary Search Tree specifically optimized for quick lookups using a unique identifier. Rather than separating the data payload from the structural linkage, the system uses an elegant architecture where `Book.java` acts as both the object-oriented business entity and the tree node.

3.1 Node Object Design & Field Layout

Each instance of the `Book` class maintains encapsulated parameters representing its bibliographic values, alongside a set of references that represent the left and right paths within the tree structure:

- **private int isbn:** The unique International Standard Book Number. This primitive integer serves as the exclusive sorting key used across all tree balancing comparisons.
- **private String title / private String author:** The bibliographic metadata payload stored directly at the node.
- **private Book left / private Book right:** Self-referential node pointers targeting subtrees containing books with smaller or larger keys, respectively.

3.2 Recursive Insertion Optimization

The arrangement of the tree is maintained dynamically during record insertion via the `recursiveInsert(Book currentNode, Book newBook)` mechanism. This method employs a divide-and-conquer branching paradigm:

- **Duplicate Verification Base Case:** Before any subtree injection occurs, the system matches `newIsbn == currentIsbn`. Upon intercepting a match, it rejects the insert command with an error message, preserving the absolute key integrity of the index.
- **Left Subtree Traversal:** If the incoming ISBN is smaller than the current node's value, the system inspects `currentNode.hasLeft()`. If true, it recurses downward; if false, it links the new node into that available space.
- **Right Subtree Traversal:** If the incoming ISBN exceeds the current node's value, it mirrors the left side logic on the right branch.

3.3 Logarithmic Search Path Operations

The main benefit of this catalogue design is its quick search capability, developed within `recursiveSearch(Book currentNode, int isbn)`. By comparing the search key against the current node's ISBN, the algorithm completely rules out half of the remaining search space at every step. This leads to an average time complexity of $O(\log n)$. The function relies on two clear base cases and two recursive directions:

- **currentNode == null:** Reached a leaf dead-end; returns null immediately, signifying the book does not exist.
- **currentNode.getIsbn() == isbn:** Key target found; returns the book reference to the caller.

- **isbn < currentNode.getIsbn():** Recurses down the left branch.
- **isbn > currentNode.getIsbn():** Recurses down the right branch.

3.4 Multi-Scenario Node Deletion (The Borrowing Trigger)

When a book is borrowed, it must be removed from the catalogue tree using the `recursiveDelete(Book currentNode, int isbn)` method. Deleting a node from a BST is complicated because the remaining elements must be re-linked to preserve the sorting properties. The system handles all three classic structural deletion scenarios:

- **Scenario 1: Removing a Leaf Node (No Children):** If the book node has neither left nor right links, the function removes it simply by returning null to its parent pointer link.
- **Scenario 2: Removing a Node with a Single Child:** If the node lacks a left branch (`!currentNode.hasLeft()`), it returns its right child reference. If it lacks a right branch (`!currentNode.hasRight()`), it returns its left child. This effectively promotes the single child up a level to replace the deleted node.
- **Scenario 3: Removing a Node with Two Children (In-Order Successor):** When a node has two branches, removing it completely requires finding its In-Order Successor – the smallest value in its right subtree.

The system invokes `getMinValueNode(currentNode.getRight())`, which tracks down the leftmost leaf of the right subtree. The values are then cloned, overwriting the data fields, and the duplicate successor node is recursively removed from the right subtree branch.

3.5 Full-Traversal Keyword Search (Title & Author)

Because the BST is ordered by ISBN rather than by bibliographic metadata, locating books by title or author requires a complete traversal of all nodes rather than a directed search path. Two methods – `searchByTitle(String keyword)` and `searchByAuthor(String keyword)` – support this via in-order recursive traversal. Both operations share the same structural logic:

- **Case-insensitive partial matching:** This is applied at every node by converting both the stored field and the keyword to lower-case before calling `contains()`.
- **Full in-order traversal:** The traversal visits all nodes in the left subtree, checks the current node, then visits all nodes in the right subtree, guaranteeing no record is skipped regardless of its position in the tree.
- **Boolean no-result flag:** A boolean flag is propagated back up the recursion chain. If no match is found across the entire tree, a descriptive no-results message is printed to the console.

Because these operations cannot exploit the ISBN-ordered structure of the BST, they carry a worst-case time complexity of $O(n)$, visiting every node exactly once. This trade-off is intentional: the design preserves fast $O(\log n)$ ISBN lookups for the common case while still supporting flexible keyword discovery when needed.

4. Transaction Registry: Stack Design

The `BorrowStack.java` class tracks user borrowing operations. Unlike a catalogue, which requires structured lookups, transaction ledgers are inherently chronological. Users care most about reviewing their recent activity. A Last-In-First-Out (LIFO) Stack perfectly mirrors this real-world workflow.

4.1 Encapsulation of the Legacy Collection Layer

To prevent unauthorized structural modifications, the system wraps Java's built-in `java.util.Stack` inside a controlled custom object wrapper. This ensures that arbitrary index shifts or mid-collection element deletions are completely blocked, exposing only the safe operations required by the library architecture.

4.2 LoanRecord: Enriched Transaction Model

The `LoanRecord` system captures context about who borrowed the book or when. The `LoanRecord` class was introduced to address both gaps:

- **private Book book:** A defensive snapshot copy of the borrowed book, created at construction time via the `private copyBook()` helper. Storing a copy rather than the live BST node ensures that subsequent catalogue mutations (such as the node deletion triggered by borrowing) cannot retroactively alter the bibliographic details recorded for the transaction.
- **private String studentId:** The identifier of the student who initiated the borrowing transaction.
- **private LocalDate borrowDate:** The date the book was borrowed. The primary constructor sets this automatically to `LocalDate.now()`, while a secondary constructor accepts an explicit date so that overdue scenarios can be simulated for demonstration and testing.
- **private boolean fineProcessed:** A one-way status flag, defaulting to false, that records whether a fine has already been charged for this loan. It is read via `isFineProcessed()` and set via `markFineProcessed()`, and exists to make fine processing idempotent.

The borrowing context of a `LoanRecord` (book, student ID, and borrow date) is immutable: it is set once at construction and exposes no setters, so a logged transaction's history cannot be retroactively altered. The single exception is the `fineProcessed` flag, which can be flipped exactly once from false to true through `markFineProcessed()` to prevent the same overdue loan from being charged more than once. The `BorrowStack` now stores `Stack<LoanRecord>` rather than `Stack<Book>`, meaning every entry on the history stack carries the full transactional context needed by the `FineManager`.

4.3 Null-Safety Boundaries & Input Defenses

One thing that can quietly break a system is letting a null reference slip through into a deeper layer. To guard against this, the stack checks what it receives before doing anything with it:

```
public void push(LoanRecord record) {
    if (record != null) {
        stack.push(record);
    } else {
        System.out.println("History Error: Cannot log an empty transaction record.");
    }
}
```

This structure forms an absolute guard rail. If an upstream search failure occurs or a corrupted entity is passed, the stack rejects the null pointer, avoiding a runtime `NullPointerException` during subsequent output rendering operations.

4.4 Non-Destructive Reverse-Chronological Rendering Logic

The transaction history needs to show the most recently borrowed book first, but iterating over a Java Stack the normal way prints things from oldest to newest. Calling `pop()` in a loop would fix the order but wipe the history in the process. To get around both problems, the `show()` method uses a reverse index loop that reads the stack from top to bottom without touching its contents:

```
for (int i = stack.size() - 1; i >= 0; i--) {
    LoanRecord record = stack.get(i);
    Book b = record.getBook();
    System.out.println("[ISBN: " + b.getIsbn() + "] " + b.getTitle() +
        " by " + b.getAuthor());
    System.out.println("    Borrowed by: " + record.getStudentId() +
        " on " + record.getBorrowDate());
}
```

This way, the most recent transaction always appears at the top of the output, and the stack itself stays completely untouched for anything that comes after.

4.5 Student-Specific Record Retrieval

The `FineManager` needs to evaluate a student's borrowing records in order to calculate how overdue their loans are. The stack therefore exposes two top-down lookup helpers that scan from the most recent record to the oldest:

- **`getAllByStudent(String studentId)`:** Collects every `LoanRecord` belonging to the student into a list, ordered most-recent-first. This is the method the Fine Manager's "Calculate Fine" option actually uses, so that each of a student's outstanding loans can be evaluated in turn.
- **`getLatestByStudent(String studentId)`:** Returns only the single most recent `LoanRecord`

for the student, or null if none exists. It is provided as a convenience lookup for callers that need just the latest transaction.

The `getLatestByStudent()` implementation returns the first match found while scanning from the top of the stack downwards:

```
public LoanRecord getLatestByStudent(String studentId) {  
    for (int i = stack.size() - 1; i >= 0; i--) {  
        LoanRecord record = stack.get(i);  
        if (record.getStudentId().equals(studentId)) {  
            return record;  
        }  
    }  
    return null;  
}
```

Because the stack is traversed from top (most recent) to bottom (oldest), both methods surface records in chronological-reverse order without requiring any sorting or secondary data structure. Each operation carries a worst-case complexity of $O(k)$, where k is the total number of records in the borrowing history.

5. Fine Management Engine

The `FineManager.java` class is a self-contained subsystem responsible for calculating overdue penalties, maintaining per-student outstanding balances, and providing payment and balance-inquiry operations through the console interface. It connects to the rest of the system through `BorrowStack`, which supplies the `LoanRecord` data needed to determine how long a book has been on loan.

5.1 Fine Calculation Logic

Fines are computed against two configurable constants declared at the top of the class:

- **FINE_PER_DAY = RM 0.50:** The penalty applied per calendar day beyond the due date.
- **LOAN_PERIOD_DAYS = 14:** The standard borrowing window. Any book not returned within this window is considered overdue.

The `calculateFine(LoanRecord record)` method adds `LOAN_PERIOD_DAYS` to the record's `borrowDate` to establish a due date, then measures the gap between that date and today using `ChronoUnit.DAYS.between()`. If the result is zero or negative, no fine applies. Otherwise, the overdue count is multiplied by `FINE_PER_DAY` and returned as a double.

5.2 Balance Tracking with HashMap

Student balances are stored in a `HashMap<String, Double>` where the key is the student's identifier string and the value is their total outstanding fine in Malaysian Ringgit. This structure provides $O(1)$ average-case lookups and updates for individual student records regardless of how many students are in the system.

The `addFine(String studentId, double amount)` method retrieves the current balance using `getOrDefault()` (defaulting to 0.0 for new students) and stores the updated total. The `processFine(LoanRecord record)` method combines `calculateFine()` and `addFine()` into a single call, and additionally enforces idempotent charging: it first checks `record.isFineProcessed()` and returns early with a notice if the loan has already been charged. Otherwise it calculates the fine, and only when the fine is greater than zero does it add the amount to the balance, call `record.markFineProcessed()`, and print a confirmation. When no fine is currently due, the record is deliberately left unmarked (so it can still accrue a fine later) and a “no fine currently due” notice is printed instead. This makes it safe for the UI layer to trigger a complete fine evaluation repeatedly without double-charging a student.

5.3 Payment & Display Operations

The `FineManager` exposes three output-facing operations accessible through the Fine Manager submenu:

- **payFine(String studentId):** Checks whether the student has an outstanding balance. If so, the full amount is printed as a payment confirmation and the balance is reset to 0.0. If the student has no record or a zero balance, a clear notice is printed instead.
- **showStudentBalance(String studentId):** Prints the current outstanding balance for a specific student, defaulting to RM 0.00 if no record exists.
- **showAllBalances():** Iterates over the entire balance map and prints only those entries with a value greater than zero. If all balances are cleared or no fines have been processed, an appropriate summary notice is displayed.

6. System Integration & Console UI Layer

`SmartLibrary.java` is where everything comes together. It connects the catalogue, the borrowing history, and the fine management engine into one cohesive system, translating whatever the user does in the terminal into the right actions across the underlying data structures. The main menu exposes seven options: Add Book, Search Book, Borrow Book, View History, Fine Manager, Demo for Overdue Record, and Exit. The first four operations together with Exit constitute the base five-option menu required by the brief; the Fine Manager and Demo for Overdue Record entries are intentional enhancements layered on top of that baseline to expose the fine-management subsystem and to make overdue behaviour easy to demonstrate, and they leave the originally required flow unchanged.

6.1 Atomic Borrowing Lifecycle Flow

The core feature of the system is the checkout transaction loop, which is encapsulated within `borrowBook(int isbn, String studentId)`. This method links the individual modules to create an atomic transaction flow:

- **Query Despatch:** Invokes the search algorithm on the BST catalogue to locate the target book record.
- **LoanRecord Construction:** If found, a new `LoanRecord` is instantiated with the retrieved Book reference and the active student identifier, automatically timestamping the

transaction to today's date.

- **History Preservation:** The LoanRecord is pushed directly to the BorrowStack history ledger.
- **Catalogue Excision:** The book record is immediately removed from the BST via recursive deletion, unlinking its nodes.
- **Fault Handling:** If the book does not exist or is already checked out, it displays a clear notice to the user, skipping the deletion and stack-push steps.

6.2 Extended Search Submenu

The search menu has been expanded from a single ISBN lookup into a three-option submenu:

- **Option 1 – Search by ISBN:** Dispatches directly to the BST's recursiveSearch() method, benefiting from $O(\log n)$ lookup performance.
- **Option 2 – Search by Title:** Accepts a partial keyword and delegates to BookBST.searchByTitle(), performing a full in-order traversal with case-insensitive matching.
- **Option 3 – Search by Author:** Accepts a partial keyword and delegates to BookBST.searchByAuthor(), performing an equivalent full traversal against the author field.

Empty keyword inputs are caught before dispatching to the BST, preventing meaningless full-tree traversals that would match every node.

6.3 Fine Manager Submenu

The Fine Manager is integrated as Option 5 in the main menu and exposes four operations:

- **Calculate Fine:** Prompts for a student ID and retrieves all of that student's LoanRecords via `BorrowStack.getAllByStudent()`. If the student has no borrowing records, a descriptive message is shown. Otherwise each record is passed in turn to `FineManager.processFine()`, which charges any outstanding overdue amount while the per-record fineProcessed flag prevents the same loan from being charged twice.
- **View Student Balance:** Prompts for a student ID and calls `FineManager.showStudentBalance()` to display their current outstanding amount.
- **View All Balances:** Calls `FineManager.showAllBalances()` to list every student with an outstanding fine.
- **Pay Fine:** Prompts for a student ID and calls `FineManager.payFine()` to settle their full balance.

6.4 Robust Type-Parsing & Error Recovery Scanners

One of the trickier parts of building a console program is handling user input that does not come in the expected format. Relying on something like `Scanner.nextInt()` is risky because typing the wrong thing can either corrupt the input buffer or crash the program entirely. To get around this, the input handling reads everything as a string first and then converts it:

```
String menuInput = sc.nextLine();
int choice;
try {
    choice = Integer.parseInt(menuInput.trim());
} catch (NumberFormatException e) {
    System.out.println("Invalid option. Please enter a number between 1-7.");
    continue;
}
```

This same approach is used everywhere the program expects a number, both in the main menu and when entering ISBNs. So if someone types something like “abc” by mistake, the program catches it, prints a short warning, and loops back to the menu cleanly without losing anything.

6.5 Overdue Record Demonstration Utility

Because real overdue fines depend on the passage of calendar time, Option 6 – Demo for Overdue Record provides a way to demonstrate the fine logic without waiting two weeks. The `addDemoOverdueRecord()` method constructs a synthetic Book (ISBN 999, “Demo Overdue Book” by “Test Author”), assigns it to student UM001, and back-dates the borrow date to twenty days before today using the secondary `LoanRecord` constructor. The resulting record is pushed onto the history stack so that running Calculate Fine for UM001 yields a predictable, non-zero overdue penalty (six overdue days beyond the 14-day loan period, i.e. RM 3.00). This utility exists purely to make the Fine Manager easy to verify during demonstrations and grading.

7. Algorithmic Complexity & Performance Analysis

One of the more important things to look at in any system like this is how it holds up as the data grows. The short answer is that it depends entirely on which structure is doing the work, so it's worth breaking down each one.

7.1 Data Structure Operation Complexity Matrix

System Operation	Underlying Structure	Asymptotic Complexity	Algorithmic Rationale & Boundaries
Add Book	BST Dynamic Insertion	Avg: $O(\log n)$ / Worst: $O(n)$	On balanced trees, each step eliminates half the remaining paths. If keys are added in strictly sorted order, the tree becomes a linear linked list, increasing cost to linear time.
Search Book (ISBN)	BST Key Comparison	Avg: $O(\log n)$ / Worst: $O(n)$	Logarithmic depth ensures fast lookups by comparing keys. Performance degrades to linear if the structure becomes heavily skewed.
Search by Title / Author	Full BST Traversal	Strictly $O(n)$	BST ordering cannot be exploited for non-key fields. Every node must be visited to guarantee a complete result set.
Borrow Book	BST Search + Delete + Stack Push	Avg: $O(\log n)$	Combines a search lookup, tree re-linking, and a constant-time stack push. The logarithmic search steps dominate the overall operation cost.
View History	Sequential Stack Traversal	Strictly $O(k)$	Requires rendering every borrowed entry exactly once, where k represents the current size of the checkout ledger.
Retrieve Student Records	Reverse Stack Scan	$O(k)$	<code>getAllByStudent()</code> / <code>getLatestByStudent()</code> scan the history from newest to oldest, where k is the total number of stored loan records.
Calculate Fine	Date arithmetic + HashMap update	$O(1)$ per record	Date arithmetic and HashMap lookups are constant-time operations independent of the number of books or students in the system.

System Operation	Underlying Structure	Asymptotic Complexity	Algorithmic Rationale & Boundaries
Lookup Student Balance	HashMap get()	O(1) average	Direct key-based access into the balance map; no iteration required.
View All Balances	HashMap full iteration	O(s)	Requires iterating over all s students who have been assigned a balance entry.

8. Verification Matrix & Quality Assurance Summary

To make sure everything worked as intended, the team ran the system through a full round of testing covering every feature, every input boundary, and every edge case we could think of. The goal was to be confident that nothing would break under normal use or unexpected inputs.

8.1 Comprehensive Quality Assurance Testing Ledger

Target Case	Input Payload	Expected Architectural Response	Status
Invalid Menu Option	9 or text choice	Intercepts input token, prints a numeric range warning, and safely loops back without crashing.	PASS
Non-Integer ISBN	abc	Catches NumberFormatException, prints a validation notice, and returns to main prompt.	PASS
Empty Metadata	Blank title/author	Trims input string, identifies length as zero, and rejects record addition.	PASS
Valid Book Addition	101, Java Basics, Liang	Constructs a new Book node and successfully injects it into the matching BST path.	PASS
Duplicate ISBN Check	101 again	Identifies existing matching key and blocks duplicate insertion to protect data integrity.	PASS
Search by ISBN (found)	101	Traverses branches, finds key match, and renders complete bibliographic fields.	PASS
Search by ISBN (missing)	999	Reaches a null node pointer reference and returns a clean 'Book not found' message.	PASS
Search by Title (partial)	"java"	Performs full BST traversal, matches title containing keyword case-insensitively.	PASS

Target Case	Input Payload	Expected Architectural Response	Status
Search by Author (partial)	"liang"	Performs full BST traversal, matches author field containing keyword case-insensitively.	PASS
Search by Title (no match)	"xyz123"	Completes full traversal, prints 'No books found with title containing: xyz123'.	PASS
Borrow Existing Book	101	Constructs LoanRecord, pushes to history stack, runs multi-scenario node deletion on the BST catalogue.	PASS
Borrow Missing Book	999	Fails initial search query, displays an error message, and skips stack mutation steps.	PASS
View Empty History	Query prior to checkout	Identifies stack size as zero and prints a clean 'History is empty' summary notice.	PASS
View LIFO History	Checkout order: 101, 102, 103	Iterates backwards from index size-1, rendering item 103 first, then 102, then 101 with student ID and date.	PASS
Demo Overdue Record	Menu option 6	Pushes a synthetic LoanRecord for UM001 back-dated 20 days, ready for fine evaluation.	PASS
Calculate Fine (overdue)	Student with 20-day-old borrow	Computes 6 overdue days, adds RM 3.00 to the student balance and marks the record processed.	PASS
Calculate Fine (on time)	Student with same-day borrow	Returns 0.0 fine and prints 'No fine currently due for student: <id>'.	PASS
Calculate Fine (duplicate)	Re-run on already-charged record	Detects the fineProcessed flag and prints 'Fine has already been processed' without double-charging.	PASS
View Student Balance	Known student ID	Retrieves and displays the current outstanding balance from the HashMap.	PASS
View All Balances (outstanding)	Multiple students with fines	Lists all students with balance > 0; suppresses zero-balance entries.	PASS
Pay Fine	Student ID with balance	Clears balance to 0.0 and confirms payment amount.	PASS

Target Case	Input Payload	Expected Architectural Response	Status
Pay Fine (no balance)	Student ID with zero balance	Prints 'No outstanding balance' message without modifying the map.	PASS

9. Conclusion

The Smart Library Management System shows what's possible when you match the right data structure to the right problem. By pairing Binary Search Trees with a Stack, the system handles two very different demands cleanly: catalogue lookups stay fast no matter how many books are added, and borrowing history always surfaces in the order that makes sense to the user.

The extensions introduced in this version deepen that foundation in three meaningful ways. The LoanRecord model transforms borrowing transactions from simple book references into structured, timestamped events tied to specific students – a necessary step for any system that needs to reason about time. The FineManager builds directly on this enriched record model, turning what would otherwise be a manual accounting task into an automated, $O(1)$ -per-student financial ledger with idempotent charging. And the title and author search operations make the catalogue genuinely discoverable to users who don't already know a book's ISBN, accepting the $O(n)$ traversal cost as a reasonable trade-off given the infrequency of such queries relative to standard ISBN lookups.

Beyond the data structures themselves, the decision to enforce an ADT interface and build in proper input validation meant the system stayed predictable and didn't fall apart at the edges. Ultimately, this project reinforced a simple but easy-to-overlook idea: getting the data structure right from the start is what makes everything else easier to build on top of.